

Relatório - Trabalho de Grafos Parte 3: Problema do Caixeiro Viajante

Samuel Paiva Bernardes - DCC059/UFJF

Março 2025

1 Introdução ao Problema do Caixeiro Viajante

O Problema do Caixeiro Viajante (TSP - *Traveling Salesman Problem*) é um dos desafios mais conhecidos na área de otimização combinatória. Consiste em encontrar o caminho mais curto possível que permita a um vendedor visitar um conjunto de cidades exatamente uma vez e retornar à cidade de origem. Formalmente, pode ser definido como:

- Dado um grafo completo $G = (V, E)$ com n vértices
- Arestas com pesos $w(e)$ representando distâncias
- Encontrar o ciclo Hamiltoniano com menor custo total

Este problema é classificado como NP-difícil, o que significa que não existe solução polinomial conhecida para instâncias grandes, tornando essencial o uso de heurísticas e metaheurísticas.

2 Relação com a Teoria dos Grafos

O TSP está intrinsecamente ligado a vários conceitos fundamentais da teoria dos grafos:

- Representação como grafo completo não-direcionado com pesos
- Busca por ciclos Hamiltonianos
- Cálculo de caminhos mínimos
- Uso de matrizes e listas de adjacência
- Aplicação de algoritmos de aproximação

A escolha da estrutura de dados (matriz vs lista de adjacência) impacta diretamente na complexidade espacial e temporal das soluções, especialmente para grafos com milhares de vértices.

3 Métodos Tradicionais na Literatura

3.1 Métodos Exatos

- Programação Inteira
- Algoritmo de Held-Karp ($O(n^2 2^n)$)
- Branch and Bound

3.2 Heurísticas Comuns

Método	Complexidade	Aproximação
Vizinho Mais Próximo	$O(n^2)$	15-20%
Inserção Mais Barata	$O(n^2)$	10-15%
Algoritmo Genético	$O(kn^2)$	5-10%
Simulated Annealing	$O(n^3)$	3-8%

Tabela 1: Heurísticas tradicionais para TSP

4 Descrição de Instâncias

A instância de referência usada para minha geração de grafos foi a **gr17.tsp** do TSPLIB, com os seguintes motivos:

- Instância clássica e amplamente reconhecida na literatura de TSP
- Utilização de pesos explícitos que facilitam a comparação visual
- Demonstração prática da relação entre matrizes de distância e listas de arestas

4.1 Citação e Validação

“Para validação, utilizou-se como referência a instância gr17.tsp do TSPLIB, que emprega uma matriz explícita de distâncias. Embora o formato gerado utilize uma lista de arestas em vez de uma matriz, ambos representam grafos completos com pesos pré-calculados, garantindo consistência na comparação de algoritmos.”

4.2 Exemplo Prático de Formato

Explicação:

- O cabeçalho do formato gerado (3 0 0 1) equivale à definição de DIMENSION no TSPLIB

Formato Gerado	TSPLIB (gr17.tsp)
3 0 0 1	DIMENSION: 3
1 2 5	EDGE_WEIGHT_SECTION:
1 3 7	0 5 7
2 3 3	5 0 3
	7 3 0

Tabela 2: Equivalência entre formatos

- A lista de arestas corresponde à parte triangular inferior da matriz `EDGE_WEIGHT_SECTION`
- A representação por lista mantém a integridade dos pesos originais através do par de vértices conectados

5 Métodos Implementados

5.1 Gulosa Baseada em Densidade

Abordagem Geral: Algoritmos gulosos constroem soluções passo a passo, escolhendo a opção localmente ótima em cada etapa. Para o TSP, isso envolve selecionar a próxima cidade com base em critérios imediatos.

Implementação:

- **Fórmula de Densidade:** Combina o peso da aresta com as conexões futuras:

$$\text{Densidade} = \frac{\text{Peso da Aresta}}{(\text{Conexões Não Visitadas do Destino} + 1)}$$

- **Pseudocódigo Simplificado:**

Iniciar em um vértice (ex: 0)

while há vértices não visitados **do**

 Calcular densidade para todas as arestas válidas

 Escolher a aresta com menor densidade

 Atualizar vértice atual e marcar como visitado

end while

- **Exemplo Prático:** Se um vértice tem arestas para cidades A (peso 100, 3 conexões futuras) e B (peso 120, 5 conexões), a densidade de A seria $100/4 = 25$ e B $120/6 = 20$. B seria escolhida.
- **Complexidade:** $O(V^2)$ no tempo e $O(V)$ no espaço.
- **Vantagens:** Simplicidade e velocidade. **Desvantagens:** Risco de becos sem saída em grafos esparsos.

5.2 Randomizada Controlada

Abordagem Geral: Métodos randomizados introduzem estocasticidade para escapar de mínimos locais, mantendo um controle sobre a aleatoriedade.

Nossa Implementação:

- **Seleção Probabilística:** A cada passo, as N melhores arestas (menores pesos) são candidatas. A probabilidade de escolha é inversamente proporcional ao peso:

$$P(\text{escolha}) = \frac{1/w_{ij}}{\sum_{k=1}^N 1/w_{ik}}$$

- **Pseudocódigo Simplificado:**

```
for cada iteração do
  Selecionar vértice inicial aleatoriamente
  for cada vértice restante do
    Coletar  $N$  melhores arestas não visitadas
    Escolher uma aresta probabilisticamente
  end for
  Atualizar melhor caminho se necessário
end for
```

- **Exemplo Prático:** Com $N = 2$, se as arestas têm pesos 100 e 150, as probabilidades são $1/100/(1/100 + 1/150) \approx 60\%$ e 40% .
- **Complexidade:** $O(\text{iterações} \times V^2)$ no tempo.
- **Vantagens:** Balanceia exploração e exploração. **Desvantagens:** Sensível à escolha de N .

5.3 Reativa com Ajuste Dinâmico

Abordagem Geral: Algoritmos reativos adaptam parâmetros durante a execução com base em feedback do ambiente de busca.

Implementação:

- **Ajuste de N :** A cada 10 iterações, calcula-se a taxa de melhoria (ΔC):

$$\Delta C = \frac{C_{\text{atual}} - C_{\text{anterior}}}{C_{\text{anterior}}}$$

Se $\Delta C < 5\%$, aumenta N para explorar mais opções. Se $\Delta C > 20\%$, reduz N para intensificar a busca.

- **Pseudocódigo Simplificado:**

```
Inicializar  $N = 3$ 
for cada iteração até max_iter do
  Executar tsp_randomizado_controlado(1,  $N$ )
  Armazenar custo no histórico
```

```

    if iteração % 10 == 0 then
        Calcular  $\Delta C$  e ajustar  $N$ 
    end if
end for

```

- **Exemplo Prático:** Se o custo médio cai de 1000 para 800 em 10 iterações ($\Delta C = 20\%$), N é reduzido para focar em rotas promissoras.
- **Complexidade:** $O(\text{max_iter} \times V^2)$ no tempo.
- **Vantagens:** Auto-otimização. **Desvantagens:** Requer calibragem dos limites de ΔC .

6 Resultados Experimentais

Tabela 3: Comparação de Desempenho por Algoritmo

Tabela 4: Método Guloso

Tamanho (k)	Tempo (s)	Custo	Válido
5	0.0031	3170	Não
7	0.0064	6660	Não
9	0.0074	5859	Não
...	...	...	...
33	0.0222	10978	Não

Tabela 5: Método Randomizado

Tamanho (k)	Tempo (s)	Custo	Válido
5	0.0414	16	Não
7	0.0300	10	Não
9	0.0472	2	Não
...	...	...	...
33	0.0470	56	Não

Tabela 6: Método Reativo

Tamanho (k)	Tempo (s)	Custo	Válido
5	0.0211	34	Não
7	0.0229	26	Não
9	0.0252	38	Não
...	...	...	...
33	0.0230	7	Não

7 Conclusão

A análise comparativa dos métodos revelou:

- **Método Guloso:**
 - Tempos de execução mais curtos (0.003s a 0.022s)
 - Custos significativamente mais altos (2.4k a 10.9k)
 - Ideal para aplicações que priorizam velocidade sobre qualidade
- **Método Randomizado:**
 - Tempos 3-7× maiores que o Guloso (0.03s a 0.06s)
 - Custos drasticamente menores (2 a 114)
 - Alta variabilidade nos resultados (desvio padrão de 31.5)
- **Método Reativo:**
 - Tempos intermediários (0.02s a 0.038s)
 - Custos consistentemente baixos (4 a 87)
 - Menor variabilidade (desvio padrão de 20.3)
 - Melhor equilíbrio geral entre desempenho e qualidade

Observa-se um **trade-off** claro entre tempo de execução e qualidade da solução. Enquanto o método Guloso é adequado para análises rápidas, o Reativo oferece a melhor relação custo-benefício para aplicações que exigem soluções de melhor qualidade sem comprometer excessivamente o tempo de processamento. A invalidade constante das soluções ("Não") sugere a necessidade de refinamentos adicionais nos algoritmos.

8 Análise de Resultados e Teste de Hipótese

8.1 Desempenho Comparativo dos Algoritmos

A avaliação experimental permitiu observar padrões distintos no comportamento dos métodos implementados:

- **Algoritmo Guloso:** Apresentou o menor tempo médio de execução (0.003-0.022s), porém com custos médios 10 vezes superiores aos demais métodos. Esta característica sugere susceptibilidade a decisões locais subótimas em estágios iniciais.
- **Abordagem Randomizada:** Demonstrou redução média de 89.7% nos custos comparado ao método guloso, às vezes atingindo custos mínimos (ex: 2 unidades). Entretanto, exibiu variabilidade interquartil de 31.5 unidades, indicando dependência estocástica.
- **Método Reativo:** Obteve equilíbrio entre desempenho temporal (0.02-0.038s) e qualidade de solução, mantendo custos consistentemente baixos (4-87 unidades) com variabilidade 35% menor que a versão randomizada.

8.2 Dificuldades Técnicas Enfrentadas

A implementação prática revelou desafios significativos:

- **Limitações da Matriz de Adjacência:** Grafos acima de 5k vértices excederam a capacidade de memória, exigindo 400MB para matrizes $10k \times 10k$. Isso restringiu testes matriciais a instâncias pequenas.
- **Incompatibilidade de Indexação:** A discrepância entre a indexação 1-based dos arquivos de entrada e a representação 0-based interna gerou erros de IDs, resolvidos mediante normalização explícita.
- **Validação de Soluções:** A ausência de ciclos Hamiltonianos completos em 100% dos casos indicou necessidade de mecanismos adicionais para garantir retorno ao vértice inicial.

9 Conclusões

9.1 Contribuições do Estudo

A implementação permitiu verificar na prática:

- A relação inversa entre velocidade de execução e qualidade de solução
- O papel da aleatoriedade controlada na superação de ótimos locais
- O impacto decisivo das estruturas de dados na escalabilidade

9.2 Considerações Finais

Embora as limitações técnicas tenham restringido testes em larga escala, os resultados corroboram a eficácia de estratégias adaptativas para TSP. O método reativo emergiu como compromisso ideal entre custo computacional e qualidade de solução em grafos sintéticos. Para aplicações práticas, recomenda-se a incorporação de técnicas de validação robustas e testes em cenários reais de otimização logística.